

ITI AI-Pro

Natural Language Processing



RULES

CONTENTS

➤ **Advanced Text Representation Cont.**

- Cont. Word Embeddings
- N-grams Word Embeddings
- Word Space

➤ **Sequential Modeling**

- RNN
- GRU
- LSTM

Part I

Advanced

Text Representation



GLOVE VS WORD 2 VEC

	GloVe	Word 2 Vec
Contextual Info.		
Performance	Same	Same
Criteria	Count-based Dimensionality Reduction	NN

[More](#)

LIMITATIONS

- Out-of-Vocab (OOV)
- Morphology
- Same Words with different contexts
 - He is a **kid**.
 - Do you **kid** with me?

N-GRAM WORD EMBEDDINGS



How!!!!



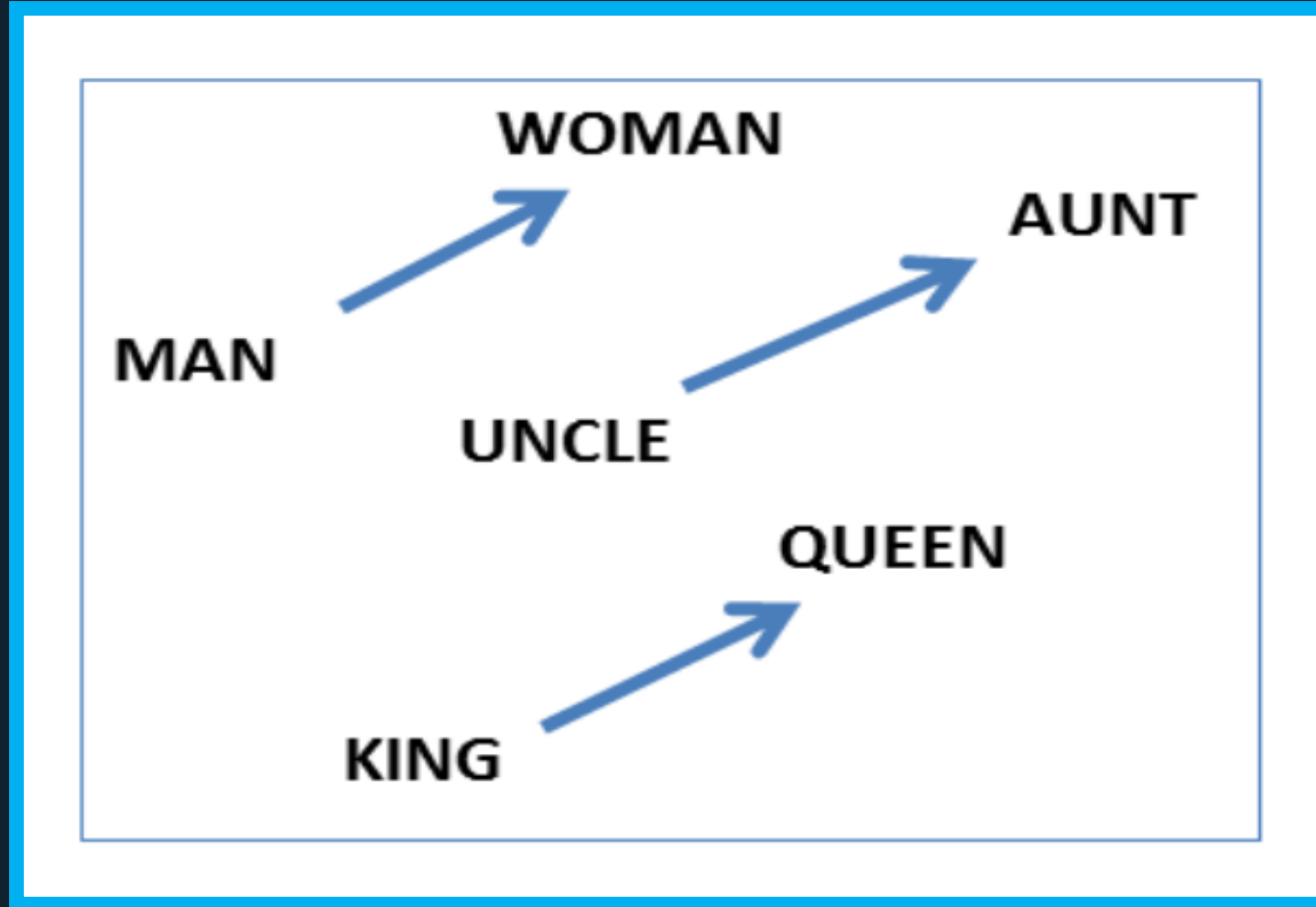
N-GRAM WORD EMBEDDINGS

- **1D CNN** to simulate n-gram behaviour

this →	0.2	0.4	-0.3
movie →	0.1	0.2	0.6
has →	-0.1	0.4	-0.1
amazing →	0.7	-0.5	0.4
diverse →	0.1	-0.2	0.1
characters →	0.6	-0.3	0.8

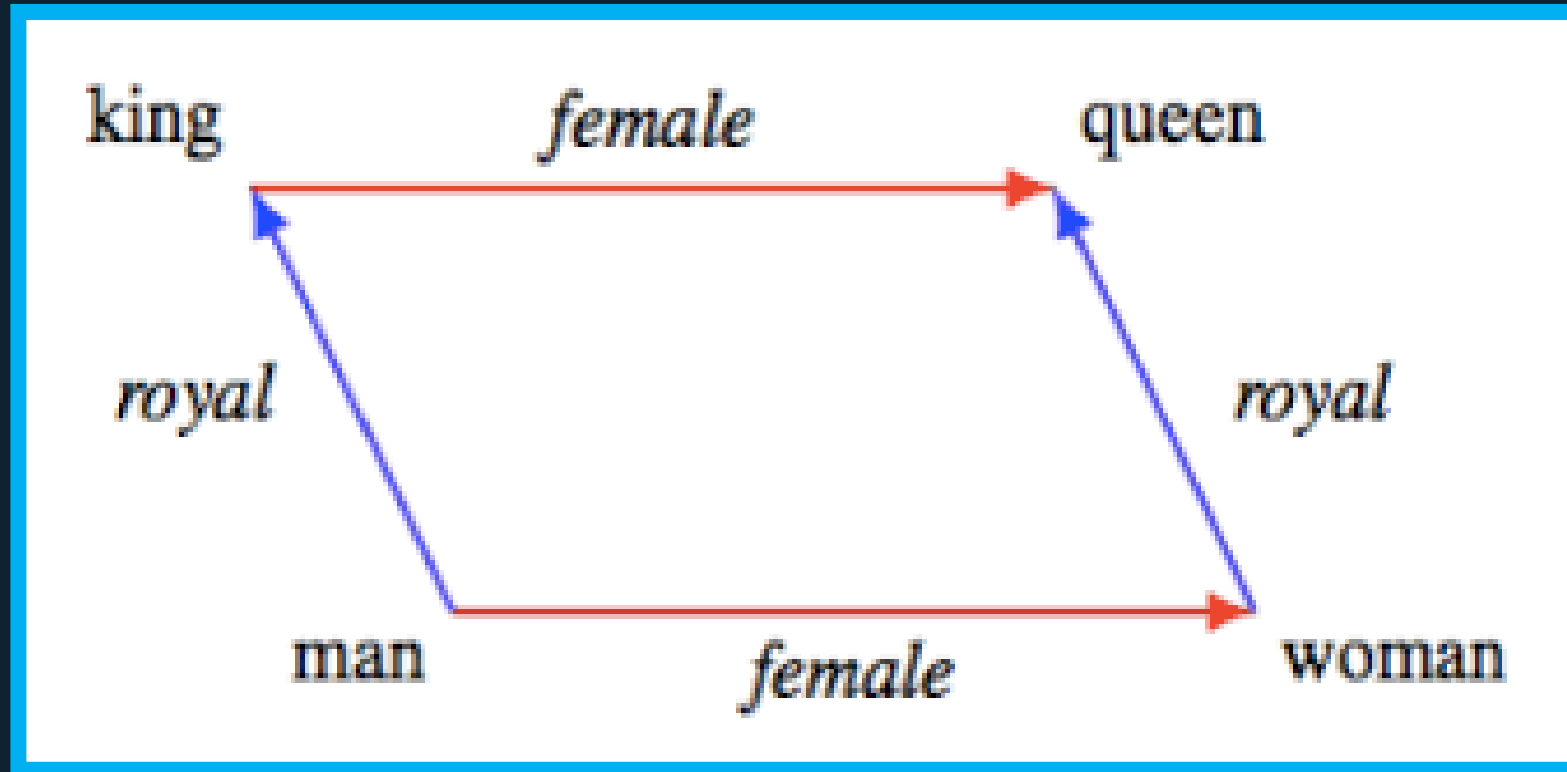
[More](#)

WORD SPACE



WORD SPACE

- Man – Woman = King – Queen



NB I Word Embedding



NB II
Word
Space



NB III

N-Grams

Word Embedding



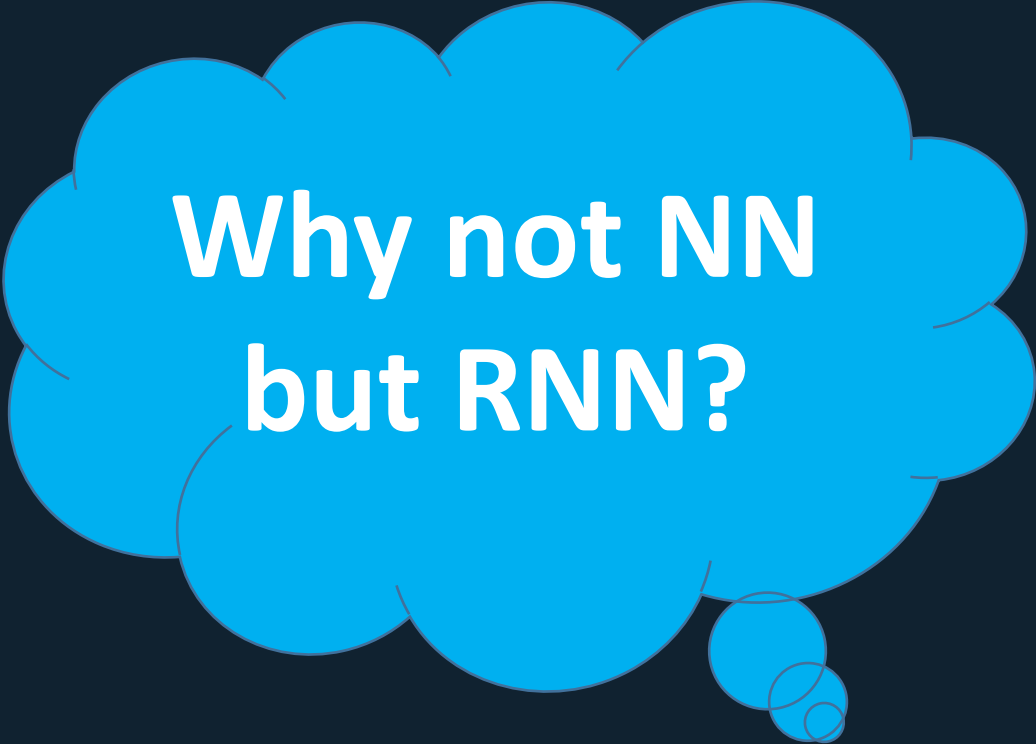


Part II

Sequential Modeling



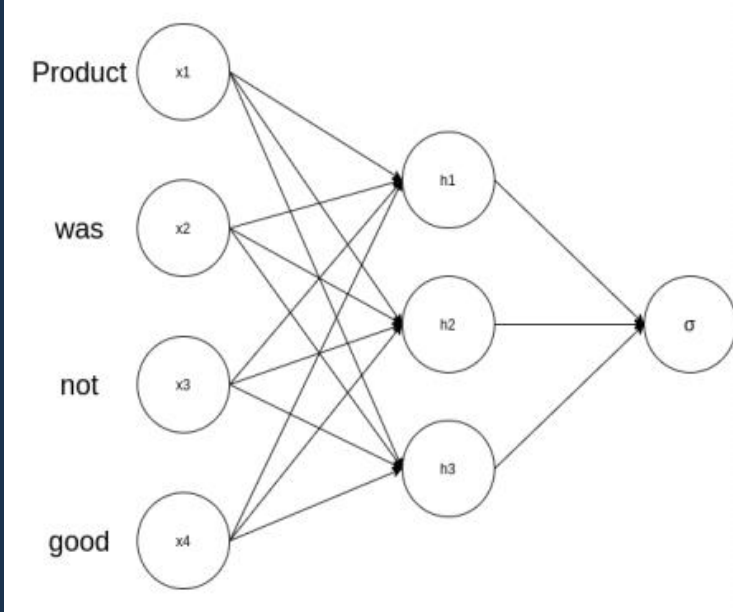
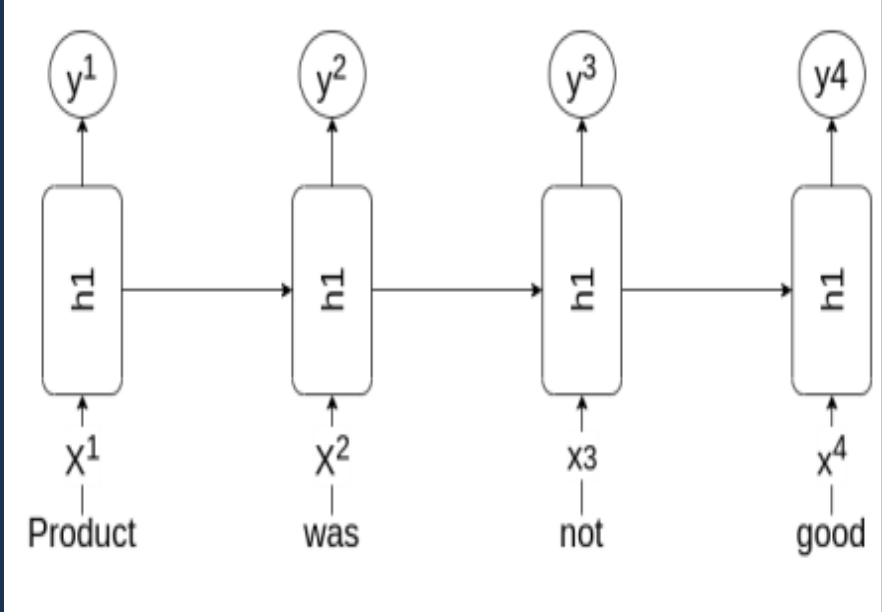
SEQUENTIAL MODELING



**Why not NN
but RNN?**

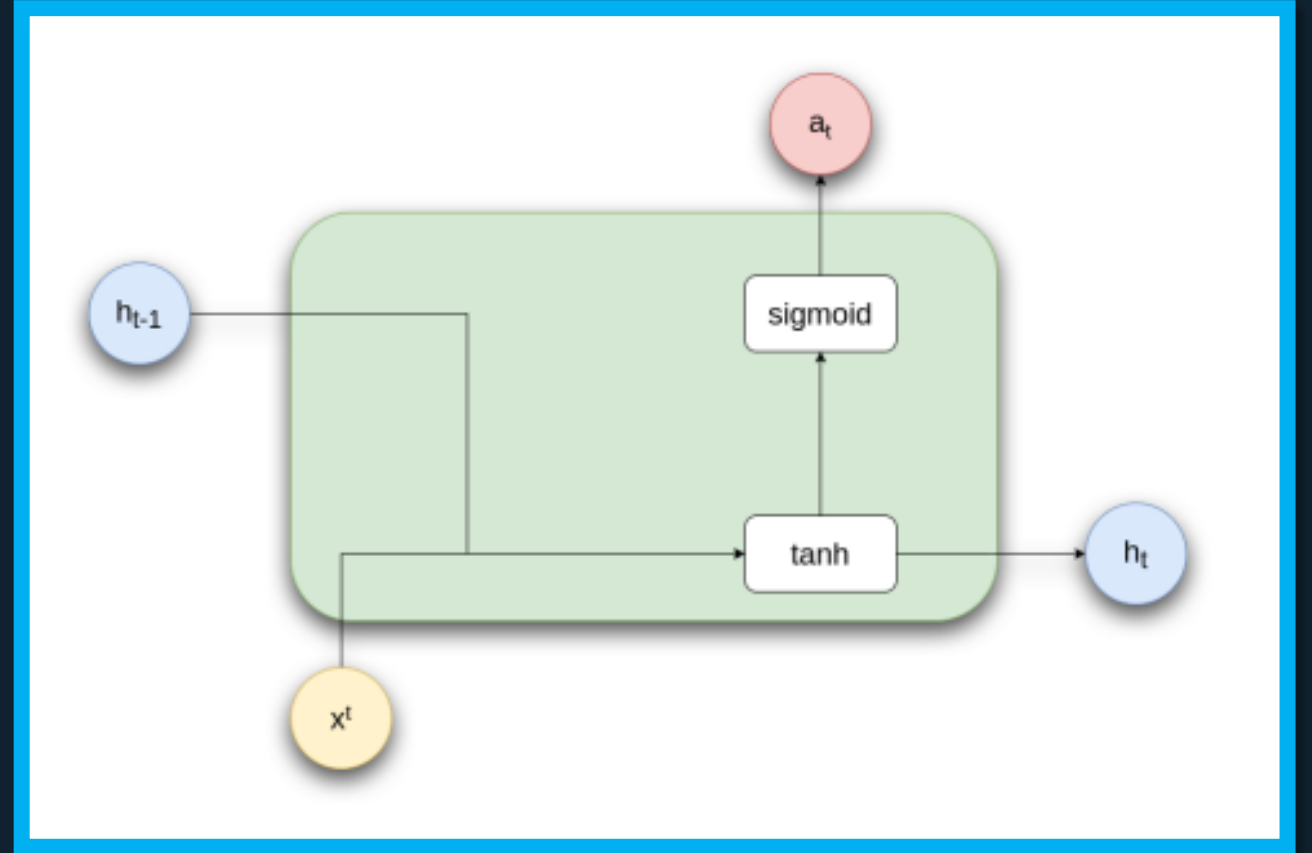
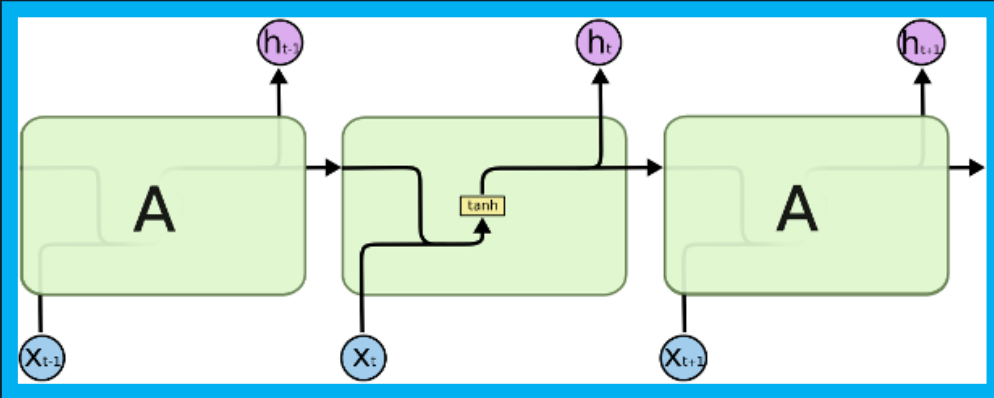


SEQUENTIAL MODELING

	FFNN	RNN
Architecture		
Sequential Info.		
Order / Position		

RNN

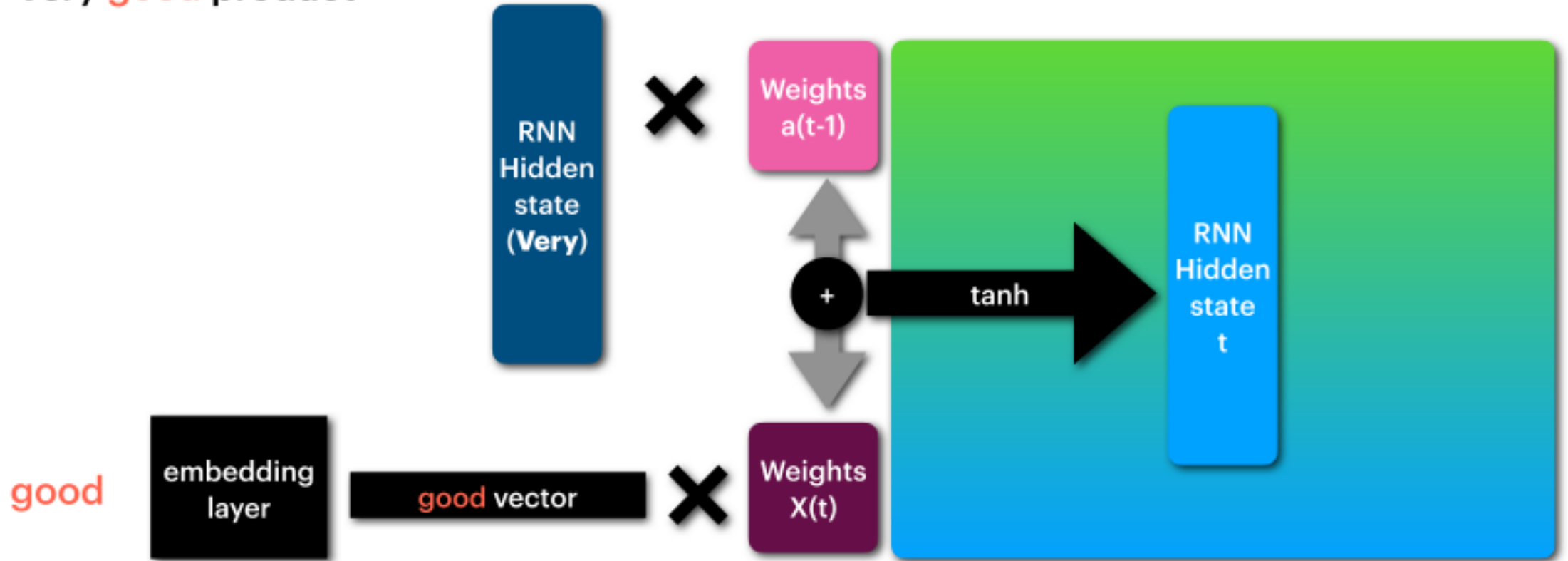
Structure



RNN

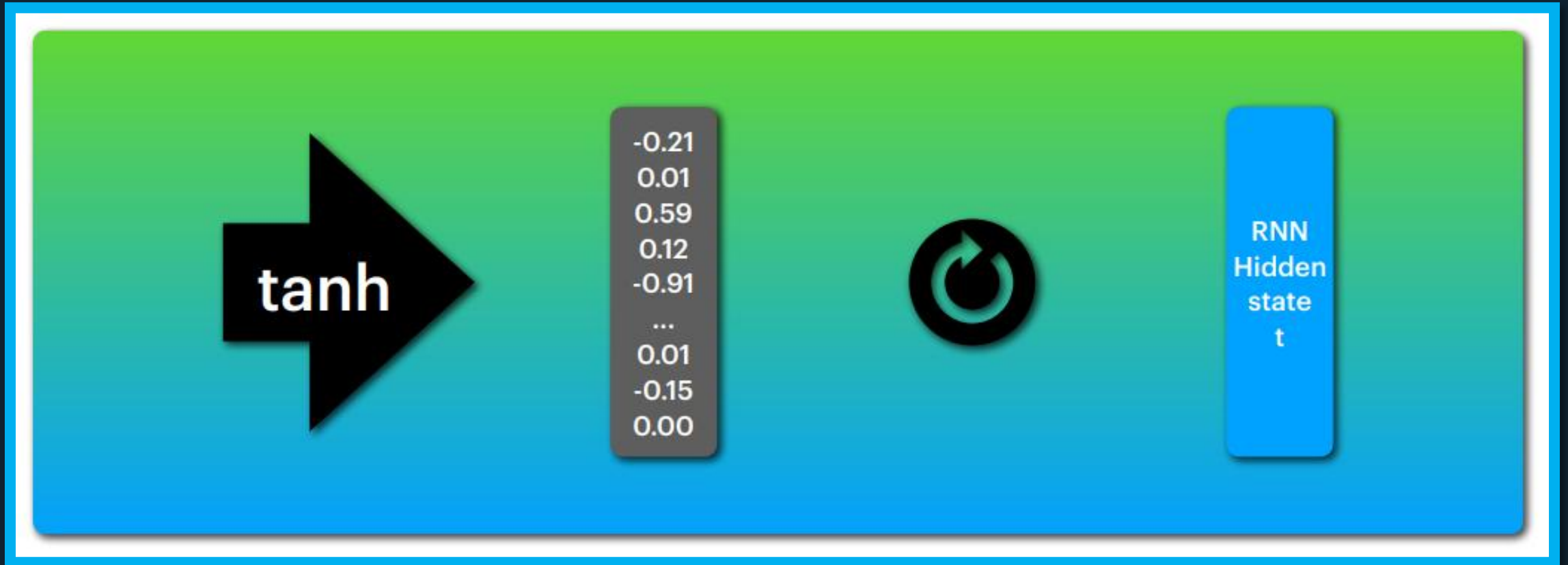
Working KH

Very **good** product



RNN

update current state based on input and previous state



Long Sequences

+

Vanishing GD

GRU

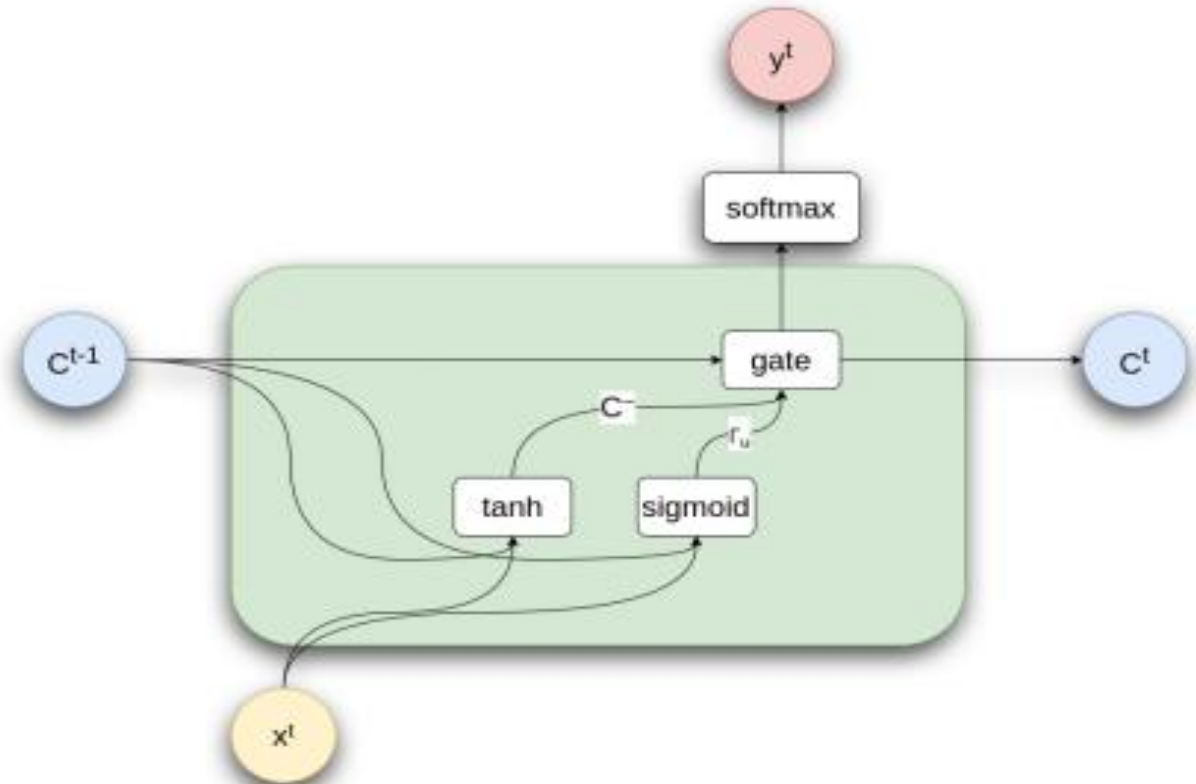
2 Gates:

- Update gate
- Reset gate

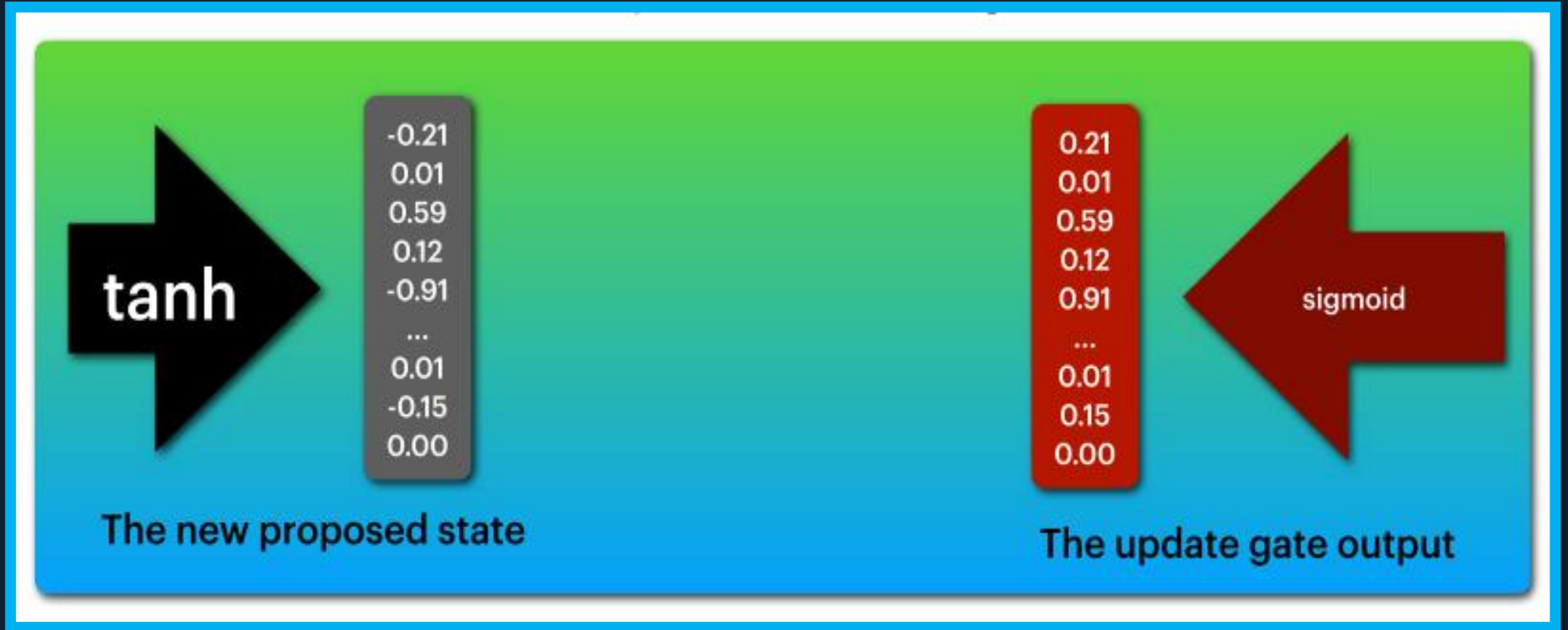
$$\tilde{C} = \tanh(W_c[C^{t-1}, X^t] + b_c)$$

$$\Gamma_u = \sigma(W_u[C^{t-1}, X^t] + b_u)$$

$$C^t = \Gamma_u * \tilde{C} + (1 - \Gamma_u) * C^{t-1}$$

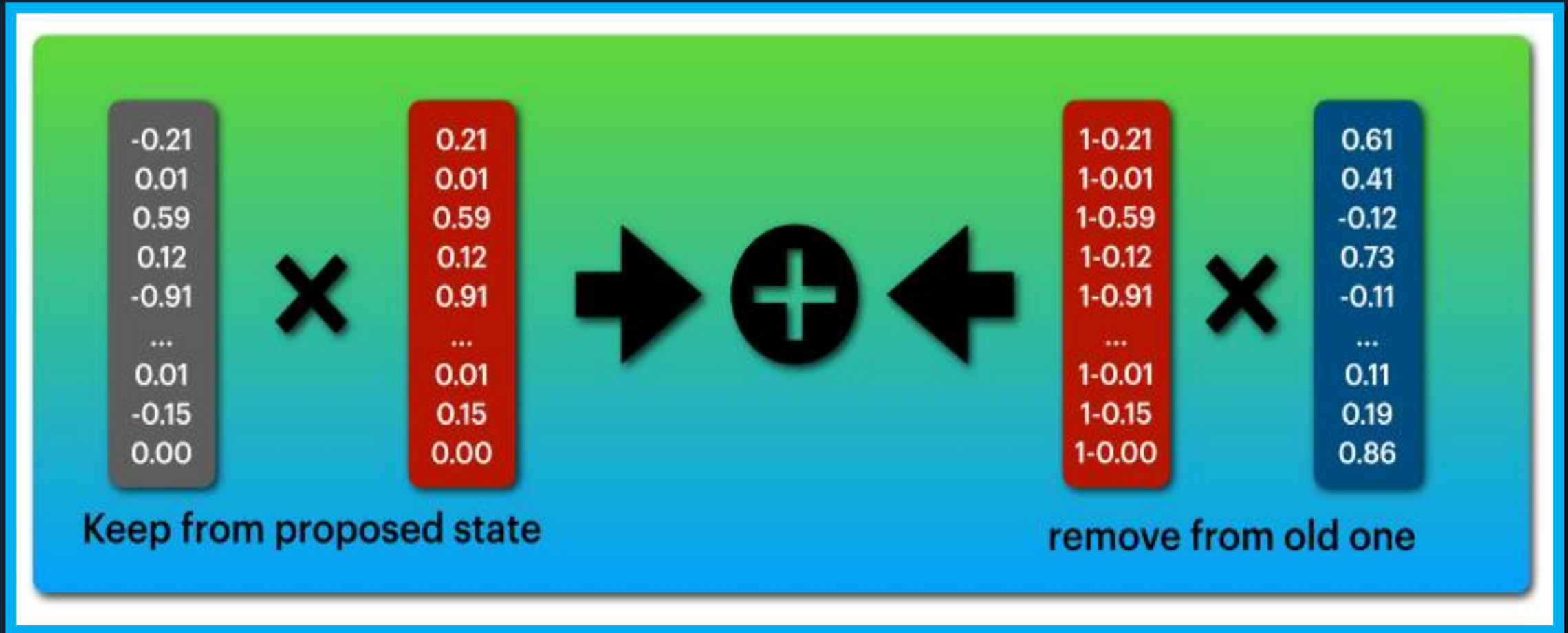


Working KH



GRU

keep and remove from memory --> better memory



Still Longer Sequences

+

Exploding GD

LSTM

3 Gates:

- **Forget gate**
keep/delete from the old long-term memory
- **Input/Update gate**
keep/delete from the new long-term memory
- **Output gate**
control the new short-term memory

LSTM

$$\Gamma_f = \sigma(W_f[a^{t-1}, x^t] + b_f)$$

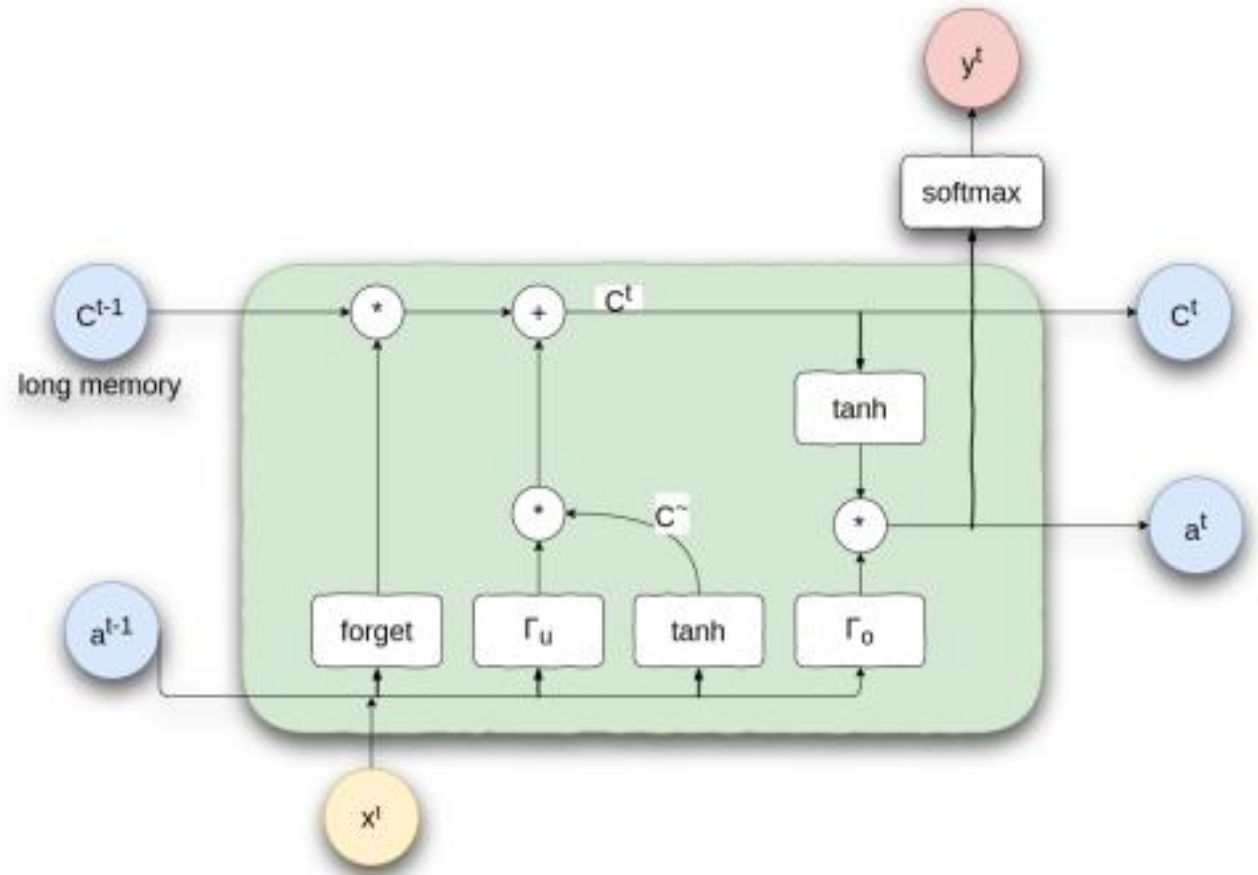
$$\Gamma_u = \sigma(W_u[a^{t-1}, x^t] + b_u)$$

$$\Gamma_o = \sigma(W_o[a^{t-1}, x^t] + b_o)$$

$$\tilde{C} = \tanh(W_c[a^{t-1}, x^t] + b_c)$$

$$C^t = \Gamma_u * \tilde{C} + \Gamma_f * C^{t-1}$$

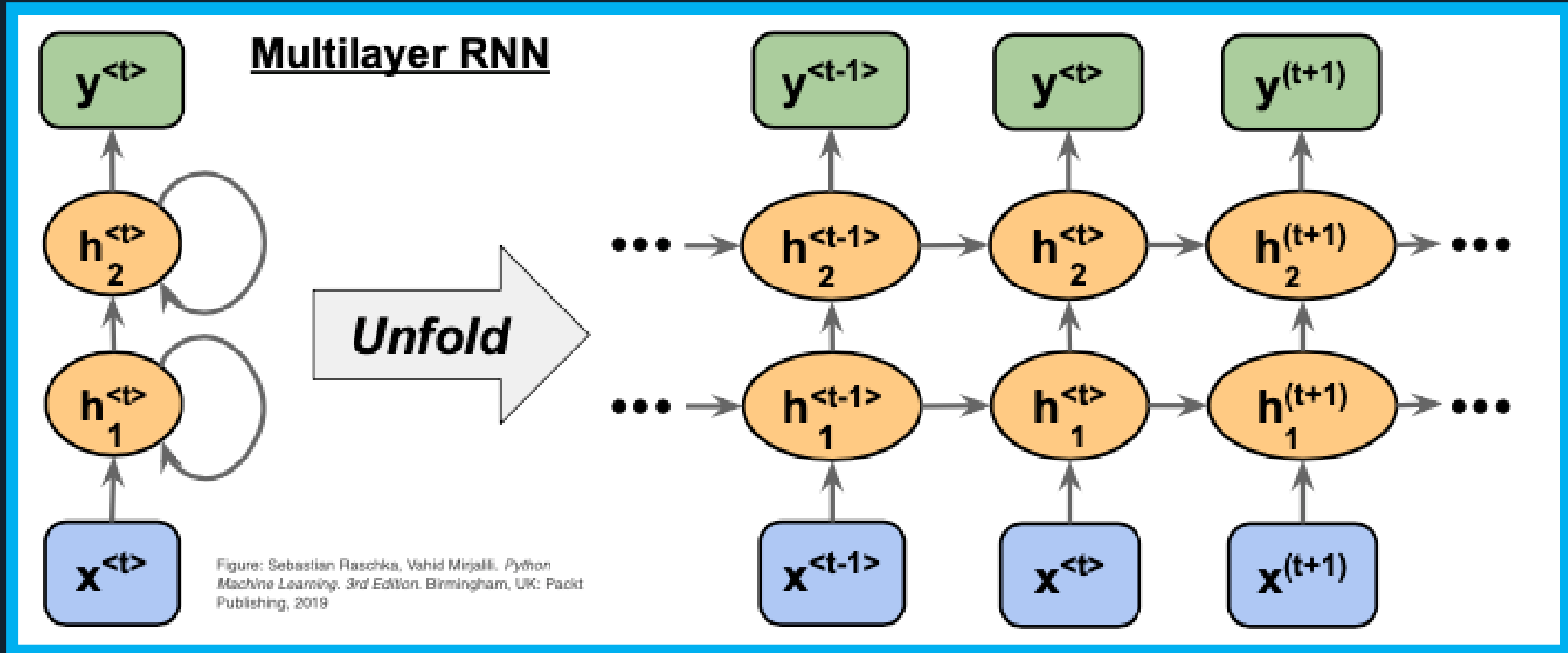
$$a^t = \Gamma_o * \tanh(C^t)$$





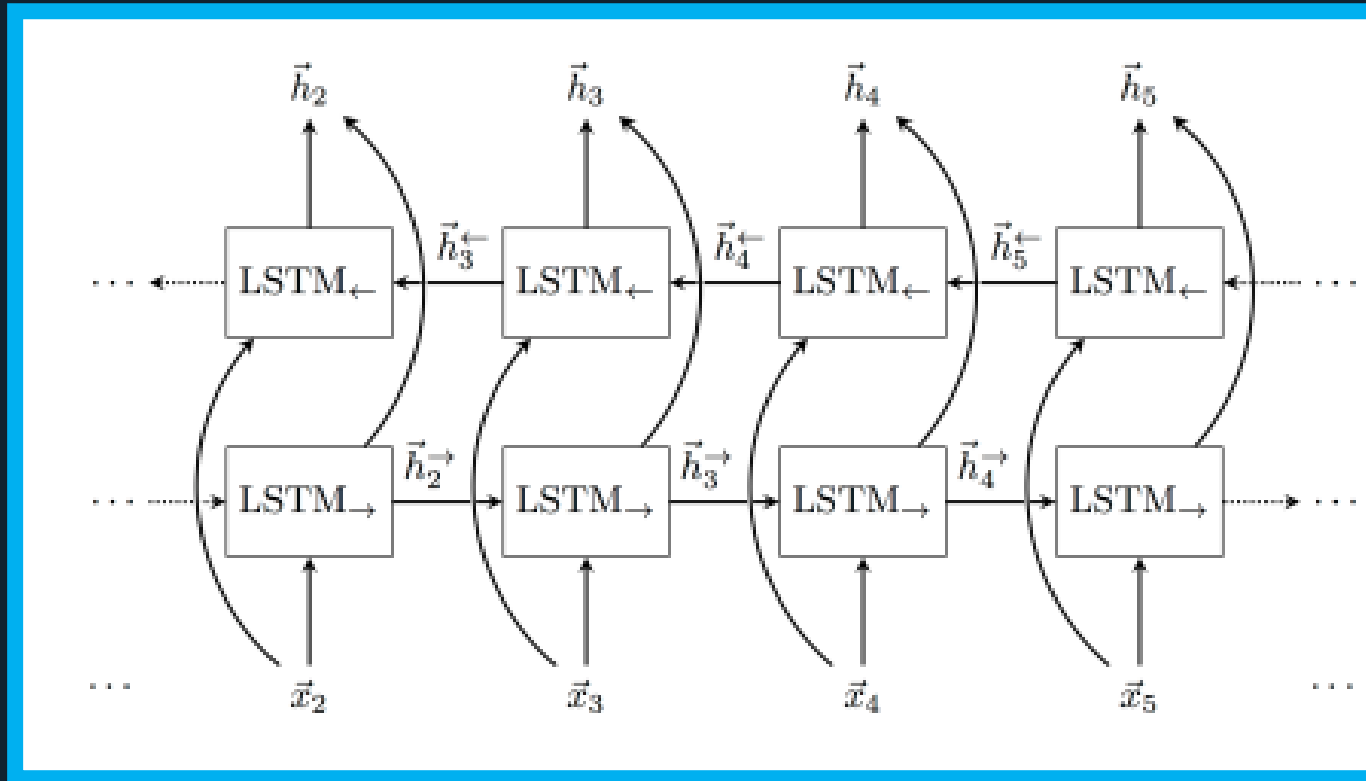
MULTI-LAYERS OF RNN

Each cell will have to **wait** for **all previous cells** in all layers

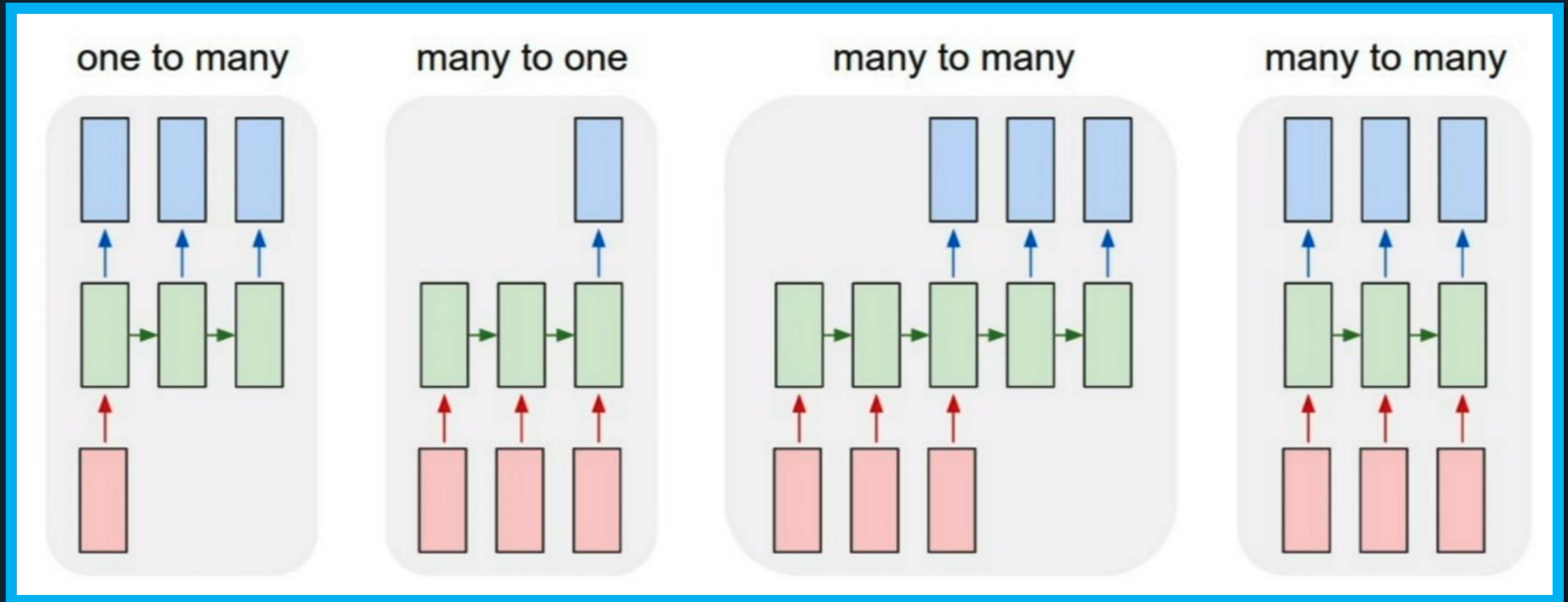


BI-DIRECTIONAL RNNs

- See in **both directions** past and forward
- Useful when **the future** can affect the current



RNNs APPLICATIONS





NB V

Text Classification With RNNs

jupyter

NB VI

**Named-Entity
Recognition NER**



THANKS
A LOT